

BrotherMode: The Anatomy of an AI Colleague

The philosophy, the fifteen laws, the code, the data flow, the cost, and the benchmark: a complete account of the open-source constitution that turns Claude into a disciplined teammate

Prepared for: everyone, from first-time AI users to senior engineers **Prepared by: Khalil Maaouni**
July 24, 2026, version 7

Executive summary: the case

Every team using AI to build things is about to hit the same wall, and most have already hit it without naming it. The models are extraordinary and getting better. The work around them is undisciplined. Sessions forget everything overnight. Parallel helpers overwrite each other in silence. Effort is spent with no sense of proportion. Claims of done arrive without proof. The same mistake gets paid for twice. None of that is fixed by a better model, because none of it is a thinking problem.

BrotherMode is the fix, and it is deliberately unglamorous: a written constitution that a Claude Code session reads and obeys, a memory vault that outlives every session, and a small toolchain that measures what actually happened. It is free, open source, and installs in ten minutes. There is nothing to learn: you delegate in plain sentences, exactly as you would to a competent contractor, and the constitution is the pre-negotiated professional standard that makes delegating safe without study.

Who it is for, stated plainly. This is built for the solo founder and the individual top performer, the person who does most things alone and occasionally shares with a small team. A company has a security officer, a data scientist, a project lead, and someone who remembers last month's decisions; working alone you have none of them, and BrotherMode gives you those specialists and a memory that survives every interruption. It scales down to one person on purpose, not up to an organization. It is not an enterprise control plane: no distributed lock service, no multi-machine coordination, no org governance. Graded against those it is a fish judged on tree-climbing. For mechanical enforcement pair it with GitHub protected branches and the CI it ships; for coordination, you are one person, which is the point.

The case rests on four claims, each expanded and sourced later in this document.

- **It removes the four largest hidden costs of AI-assisted work.** Re-exploration, because memory survives. Rework, because collisions cannot happen and claims are verified

before they compound. Wrong-direction work, because plans carry kill criteria written before the work starts. And verification overhead, because every close hands you one check you can run yourself in under a minute.

- **It answers the security question before you ask it, including the awkward part.** The skill makes zero network calls and writes only to your disk. It does create one local file holding short excerpts of your own corrections, which is how it learns; secret-shaped text is redacted, the file is owner-only, and one command purges it. Under Anthropic's commercial terms, code and prompts sent through Claude Code are not used for model training. Part VI gives the verbatim quotes, the retention periods, the residual flows that stay on by default, and the questions this document cannot answer for your security team.
- **It is cheap relative to the thing it protects.** Anthropic publishes an average of roughly 13 dollars per developer per active day across enterprise deployments. Part VII gives you the arithmetic to work out your own break-even with your own numbers rather than trusting anyone's marketing.
- **It improves itself on evidence, and reverts what does not work.** Every change to the rulebook must name the measurement it should move; the next weekly review reverts it if the measurement did not move. The constitution keeps its best version, not its latest. That property is rare enough in human processes, let alone automated ones.

There is an honest limit, stated here rather than buried. This adds no intelligence. On a single self-contained question, a bare model session is faster, cheaper, and just as smart, and you should use one. The constitution earns its overhead on multi-session work, on anything with parallel agents, and on work where being wrong is expensive. Below that threshold it is ceremony, and this document will tell you so again in Part VIII.

The thesis, in one line, and it will repeat: the model brings the intelligence; the discipline is the product.

The questions you are actually asking

The question you are actually asking	The short answer, expanded later
What is it?	A written constitution plus a small toolchain that a Claude Code session obeys: memory that survives, verification that gates, coordination that cannot silently lose work.
What does it cost?	The skill is free and MIT licensed. The AI underneath is not: Anthropic publishes roughly 13 dollars per developer per active day and 150 to 250 per month across enterprise deployments (Part VI).
Does my code or data get used for training?	Not under commercial terms. Anthropic states it does not train on code or prompts sent through Claude Code for

	Team, Enterprise, API, and cloud-provider accounts unless you opt in (Part VI, verbatim quote and source).
Does this skill add any new data exposure?	It sends nothing anywhere. It does write one new local file: redacted excerpts of your own corrections, owner-only, purgeable with one command. Part VI describes it precisely rather than claiming zero.
Does it stop the AI making things up?	It cannot stop generation errors. It stops them reaching you as accepted work, by requiring a passing command before any claim of done (Part II).
How long to learn?	Ten minutes of setup. There is no syntax to learn: you delegate in plain sentences (Part o).
Why not just write a good instructions file?	An instructions file is advice. This is advice plus enforcement plus memory plus measurement. The full comparison is in Part VII.

How to read this

This document is layered on purpose. Nobody needs all of it.

If you are	Read	Time
Deciding whether to adopt	The executive summary, then Part VII	12 minutes
New to AI agents entirely	Parts o, I, II, then Part VIII	35 minutes
A data or backend engineer	Executive summary, Part VI, then 8.2 and 8.3	20 minutes
Security or compliance	Part VI in full, plus Appendix D	15 minutes
An engineer who will extend it	Parts III, IV, V, IX	60 minutes

Contents

- Executive summary: the case
- How to read this
- Part o: Quick start, from nothing to a working session
- Part I: The philosophy, five beliefs that shape everything
- Part II: Foundations, for readers new to AI agents
- Part III: The fifteen laws, one by one

- Part IV: How the laws work together
- Part V: The code, file by file
- Part VI: Security, data flow, and cost
- Part VII: The benchmark, against the base model and the alternatives
- Part VIII: Leverage, getting the most out of it
- Part IX: Operations, teams, upgrades, and troubleshooting
- Conclusion
- Appendix A: Glossary. B: Fence contract. C: Hook wiring. D: FAQ. Sources.

Part 0: Quick start, from nothing to a working session

Read this part if you want to try it before you read about it. Everything here is checkable in ten minutes.

0.1 What you need first

- A computer running macOS or Linux. Windows works through WSL, the Linux compatibility layer.
- Claude Code, installed and signed in. It needs a paid Claude plan or API access; Part VI covers what that costs.
- Python 3, already present on most machines. Check with `python3 --version`. No packages to install: the tools use only the standard library.
- git, to clone the repository.
- Optional: Obsidian, free, to view your memory vault as a linked graph.

0.2 Install, three commands

```
# 1. install the skill
git clone https://github.com/khalilmaaouni/BrotherModeUp.git \
    ~/.claude/skills/brothermode

# 2. create your memory vault from the template
cp -R ~/.claude/skills/brothermode/vault-template ~/BrotherModeVault

# 3. check it works
python3 ~/.claude/skills/brothermode/tools/bm_score.py
```

The third command should print ten checks. On a brand-new machine most of them say NO-DATA, which is the correct answer for a system with no history: these tools never invent numbers. What you should not see is FAIL. The check names are explained in Part V; you do not need them yet.

ledger-coverage	NO-DATA	0 sessions across 0 active days last 7d
schema-2-uniform	PASS	0 pre-schema-2 lines remain
cache-economy	NO-DATA	no sessions with cache fields last 7d
vault-log-per-active-day	PASS	active days without any vault session log: none
fence-hygiene	NO-DATA	set BROTHERMODE_REGISTRIES to enable
correction-latency	PASS	0 candidates total, 0 older than 7d unprocessed
budget-vs-tier	NO-DATA	no fence lines in registries touched last 7d
prediction-seals	NO-DATA	0 sealed (target >= 5)
felt-outcome-ratings	NO-DATA	0 ratings (target >= 6 for alignment 10)
review-cadence	NO-DATA	last review: never

10 checks: 3 PASS, 0 FAIL, 7 NO-DATA. LLM judge scores only the residue.

That output is real, copied from a fresh clone. Two more steps make it automatic. First, tell Claude Code the skill exists by adding these three lines to your global instructions file at `~/.claude/CLAUDE.md`, creating it if it does not exist:

```
# brothermode
When the user types /brothermode (any casing), read and follow
~/.claude/skills/brothermode/SKILL.md before doing anything else.
```

Second, wire the three hooks. They go in `~/.claude/settings.json`, which you may need to create; if the file already exists and already has a hooks block, merge these entries into it rather than replacing it. The exact JSON is in Appendix C. The hooks are what make memory and measurement automatic instead of dependent on anyone remembering.

0.3 Your first session

Open Claude Code in a project folder and type this:

```
/brothermode read this project and tell me what it does, what shape it is in, and what
you would fix first
```

What you should see, in order: one line naming how it classified the work; a short ground map, meaning what is in the repository and what state git is in; the actual answer; and a close that names what it is unsure about. Behind the scenes the session-end hook also recorded what the session cost. Check that the wiring works, after that first real session has finished:

```
# should print one JSON line: tokens, tool calls, duration
tail -1 ~/BrotherModeVault/99-System/telemetry/outcomes.jsonl

# should list a folder named after your project, next to example-project
ls ~/BrotherModeVault/10-Projects/
```

Both checks discriminate, which matters: if the hooks are not wired, the first command reports that the file does not exist, and the second shows only the example-project folder that ships with the template. A check that passes whether or not anything happened is not a check.

Tomorrow, start a new session in the same project and ask it to continue. The difference between that second session and a second session without this skill is the entire point of the system.

0.4 If something does not work

- **The skill does not trigger.** Check that `~/.claude/skills/brothermode/SKILL.md` exists, then check that the three trigger lines from step 0.2 are in `~/.claude/CLAUDE.md`. Those lines are what connect the words you type to the file.
- **No telemetry appears.** The hooks are not wired or a path is wrong. Run `sh ~/.claude/skills/brothermode/tools/bm_sessionstart.sh` by hand: it prints a twelve-line digest plus a thirteenth line telling you the weekly review has never run, which is a to-do and not an error. If it prints nothing, the clone path is wrong; if it prints correctly, the paths inside `~/.claude/settings.json` are wrong.
- **The vault is empty after a session.** Sessions below an activity floor are deliberately not recorded, so a two-message session writes nothing. Do real work and check again.
- **It feels like too much ceremony on small tasks.** It should not: the law makes simple work skip the ceremony. If it did not, say so in plain words. That correction gets captured and becomes a rule.

Part I: The philosophy, five beliefs that shape everything

Every design choice traces back to five beliefs. Ten minutes here and the rest of the system stops being a list of rules and becomes a single argument.

1.1 Discipline is the product

For two decades the constraint on software was developer time. Then, briefly, it was model intelligence. Today frontier models reason at a level that makes raw capability the commodity: everyone has access to roughly the same brilliance at roughly the same price. What teams do not have in equal measure is the process around that brilliance: memory that compounds, verification that gates, coordination that prevents loss. The differentiator moved. BrotherMode is a bet, stated openly, that the next order of magnitude in AI-assisted output comes from the harness, not the model.

1.2 The colleague posture

"You are the founder's colleague, not a tool waiting for instructions. You own outcomes."

SKILL.md, opening law

That is the opening sentence of the law, and it defines a relationship, not a feature. A tool waits, complies, and hands back output. A colleague owns the outcome: reads the record before starting, sizes the effort, tells you when you are about to make a mistake, proves what it claims, stops at the decisions that are genuinely yours, and leaves the workspace better documented than it found it. Every law in Part III is one facet of that posture made mechanical. The posture also has a hard edge the tool posture lacks: a colleague who silently complies with a known-wrong instruction has failed at the job. Pushing back, once, plainly, with a recommendation, and then executing the call, is written into the law as a duty.

1.3 Trust is an engineering property

The naive way to trust an AI is to feel reassured by its tone. The engineered way is to make every claim carry its evidence: the command that was run, its verbatim output, and a label stating whether the claim was verified by command, verified by inspection, or merely assumed.

BrotherMode legislates the engineered way. The deepest expression is the self-score cap: the system may not grade its own work above eight out of ten without named external validation. Not because modesty is a virtue, but because ungated self-assessment inflates, always, and an inflated record poisons every decision built on it.

1.4 Laws come from paid-for failures

Nothing is in the constitution because it sounds wise. The single-writer law exists because two agents once silently destroyed each other's edits. The disk-first law exists because deliverables left in temporary folders were wiped. The build-after-every-edit line exists because a one-line change once broke a build that reading had approved. This is the design method: a failure costs an afternoon, the lesson is written as a law, and the law is placed where the next session cannot avoid reading it. It has a corollary that keeps the system lean: a rule that cannot name its failure is a candidate for deletion.

1.5 The rulebook must earn its keep

Process documents rot. They grow by accretion, contradict themselves, and end up ignored. BrotherMode treats its own rulebook as a product under test: changes are bounded, each change must name the measurement it is supposed to improve, the next weekly review reverts changes that did not deliver, and the file is held near a fixed size, so a new law must merge with or displace an old one rather than pile on. The rulebook keeps its best version, not its latest. This idea, borrowed from how neural networks are trained and applied to a text file, is the most distinctive property of the system: it is a constitution with a fitness function.

Part II: Foundations, for readers new to AI agents

Five concepts carry everything. Readers who live in this world daily can skim to Part III; the glossary in Appendix A holds the full vocabulary.

2.1 From model to agent to harness

A large language model is software trained on enormous amounts of text until it predicts good next words, which at scale produces reading, reasoning, and code. A chatbot is that model behind a text box. An agent is that model given tools: the ability to run commands, edit files, and browse, which turns talk into action. Claude Code is the agent workbench from Anthropic: it runs on your computer, connects Claude to your files and terminal, and can launch helper agents that work in parallel. A harness is everything wrapped around the agent to make its work

reliable: rules, memory, measurements, gates. BrotherMode is a harness, delivered as a skill, which is simply a playbook file the agent reads and obeys.

2.2 Why raw sessions fail: the five failures

- **Amnesia.** Yesterday's session mapped your project and made decisions; today's session knows none of it and bills you to relearn it.
- **Collision.** Two helpers edit the same file; the slower one wins; the faster one's work silently vanishes.
- **Disproportion.** Five agents for a one-line question; one careless pass for the audit that deserved five. Anthropic's own published work on multi-agent research systems names overinvesting simple queries as a top failure mode.
- **Unearned confidence.** A cheerful done on work never tested; a statistic never computed. The model is not lying; it is completing a pattern.
- **Groundhog day.** The same mistake twice, because nothing wrote the lesson where the next session had to read it.

None of these is an intelligence failure, so none is fixed by a smarter model. They are failures of memory, coordination, proportion, verification, and learning. Structure problems. Part III is the structure.

2.3 The question everyone asks first: what about hallucination?

A language model can state something false with exactly the confidence it uses for something true. That is not a defect awaiting a patch; it follows from how prediction works. Anyone who has watched an AI invent a plausible function name, a plausible column, or a plausible statistic is right to distrust the output.

BrotherMode does not claim to stop it. It cannot. What it does is refuse to let it reach you as accepted work, through four mechanisms that operate after generation rather than before it.

- **Nothing is done until a command says so.** Every task carries a done-check named in advance. A confident description of success with no passing command attached is rejected, including when the system is reviewing its own work.
- **Paths, names, and figures are never typed from memory.** The law requires confirming a file path, a function name, a flag, or a column against the actual source before using it. Most invented details die right there.
- **Numbers are computed twice, independently, before you see them.** In analysis work, a headline number that has been produced once is not a result. Anything that could not be double-checked is labeled unverified at the number itself, not in a footnote.
- **Important findings face reviewers paid to kill them.** A verdict counts only if the reviewer executed a falsification: re-ran the command, reproduced the issue, re-derived the

number. Reasoning alone does not count, because reasoning is exactly what produced the plausible error.

The honest summary: with this system you will still see wrong first drafts, and you will see far fewer wrong final answers. And when something is unverified, the document or the summary says so at the point of the claim. Part VIII shows a real case where this machinery caught two genuine defects in this system's own code, hours before it was published.

Part III: The fifteen laws, one by one

The constitution, the file named SKILL.md in the repository, runs to 467 lines and carries sixteen numbered sections, numbered 0 through 15, which is why the count below is sixteen even though the shorthand everywhere is fifteen laws. This part walks every one in the same format: what it says, why it exists, what failure it prevents, and how it connects to the others. The connections matter most; Part IV assembles them into a system.

Law 0: The invocation sequence and the safety floor

What it says. Every run opens the same way, mechanically: classify the work, assign roles, read memory, map the ground, set the loop, open the state file, execute, close with a scorecard and a memory write-back. Simple work may skip the ceremony, but whenever anything will write to disk, three steps are unconditional: check what changed on the ground, register a fence, and keep state in a file. The learning loop is explicitly forbidden from ever optimizing the floor away.

Why it exists. Sessions that improvise their opening skip different things on different days, and the skipped thing is eventually the one that mattered. A fixed sequence makes the start auditable and makes forgetting structurally hard. The floor is exempt from efficiency scoring because a learning system will happily trade safety for speed if you let it grade its own shortcuts.

What it prevents. The session that starts working before discovering another agent already edited the same files, or that a previous session left work half-landed.

How it connects. The sequence pulls in almost every other law: classification feeds Laws 1 and 7, memory reading feeds Laws 12 and 13, the ground map feeds Law 5, the loop setup feeds Laws 8 and 15.

Law 1: Work-nature profiles

What it says. Work is classified into one of six natures: product build, data and financial analysis, research and strategy, content and localization, design and creative, or operations and automation. Each profile sets the default roles, quality gates, benchmarks, and memory space. Blending is allowed when a task genuinely spans two.

Why it exists. The gates that make code trustworthy say nothing about whether a translation reads natively or whether a headline number was independently recomputed. One-size-fits-all process is how analysis ships with code-style checks and no number verification.

What it prevents. Applying the wrong definition of done: shipping analysis that runs but was never second-checked, or copy that passes lint but reads like a machine.

How it connects. Profiles select which rubric Law 15 scores against and which extra laws activate, such as the independent second check on every headline number in analysis work.

Law 2: Role assignment

What it says. The session names the professional hats the task needs, one line each, from a fixed roster: architect, product head, scientist, analyst, designer, security and privacy officer, safety officer, editor, project lead. Only the roles the work needs, never all of them.

Why it exists. A named role carries a named concern. Saying the security officer hat is on makes the session actually ask security questions, and makes its absence visible when it should have been on.

What it prevents. The vague helpful-assistant posture where every concern belongs to no one and quality is whatever emerged.

How it connects. Roles are the voice of the profile from Law 1, and the scorecard in Law 15 checks whether each named role did its job.

Law 3: The decision ladder and effort tiers

What it says. Not everything deserves an agent. A ladder is climbed and stopped at the first sufficient rung: answer directly, run one search, ask the human when their one-minute answer saves an hour, work inline, dispatch one agent, dispatch a fleet. Every delegated brief declares an effort tier: T1 (one session, a handful of tool calls), T2 (two to four helpers), T3 (a coordinated fleet with divided, fenced scopes). Independent helpers launch together as one wave, never one after another. Concurrency is capped from measured evidence, not optimism.

Why it exists. Published multi-agent research and the record of this system agree on the top failure mode: spending is disproportionate to the task, in both directions. Declaring the tier before working makes proportion a checkable claim instead of a mood.

What it prevents. Five agents for a one-line question; one blind pass for the audit that deserved five; helpers launched serially that could have run in parallel; more concurrent builders than the machine can serve.

How it connects. Tiers are written into fence lines (Law 5), judged against actual spend by the telemetry (Part V), and scored for proportion at the weekly review (Law 8).

Law 4: Token budgets and economy

What it says. Spend is treated like money. Budgets size to the declared tier. Agent reports are hard-capped near 1,500 tokens: findings, paths, verbatim command output, never pasted file contents. Long sessions post spend deltas at phase boundaries. Where real telemetry is absent, a field says not measured; an invented number may never enter the record. Waiting is done through notifications, never by polling in a loop.

Why it exists. Tokens are the unit of cost and of attention. Uncapped agent returns flood the context of the coordinator with noise, and invented numbers corrupt the learning loop at its source. A measurement system is only as good as its refusal to guess.

What it prevents. Budget surprises, context wasted on pasted logs, and a learning loop trained on fiction.

How it connects. The return cap protects the context hygiene of Law 9; the no-invention rule is what makes the self-improvement of Law 8 trustworthy; the hooks in Part V make spend measurable at all.

Law 5: The single-writer law and fences

What it says. One writer per file, ever. Before any writing agent launches, a fence line is registered in the state file naming the objective, the exact files, the boundaries, the termination condition, the declared tier, a time-to-live, and a runnable done-check. Fence first, dispatch second, never the reverse. Overlapping fences mean queue, not parallel. A fence closes only with an evidence block: the exact command run and its output. Killed agents are resumed by their identity, never respawned fresh while their transcript exists, and unfinished fence work is explicitly adopted, never silently dropped.

Why it exists. Two writers in one tree collide, and the collision is silent: the slower save wins and the faster one vanishes without an error message. Convention does not prevent this; registration before dispatch does.

What it prevents. Lost work from concurrent edits, duplicate half-finished efforts, and orphaned work that dies with its agent.

How it connects. Fences live in the state file from the floor of Law 0, carry tiers from Law 3 and done-checks from Law 15, and their closure discipline feeds the recovery guarantees of Law 9.

Law 6: The research doctrine

What it says. Research depth is decided by what would change the decision; research that cannot change it is skipped. Decision-carrying questions get searches from multiple angles run in parallel, and the hunt stops when two angles return nothing new. Sources have a hierarchy: primary evidence and official documentation beat reputable secondary sources beat forums. Version-sensitive facts such as prices, APIs, and policies are verified against current sources

every time; memory is never trusted for them. Claims delivered carry the address of a page actually opened, not one seen in a search snippet.

Why it exists. The memory of a language model is a snapshot that ages invisibly. It will state last year's price with this year's confidence. The doctrine converts research from recall into verification.

What it prevents. Confident stale facts, single-sourced claims dressed as consensus, and decisions built on a search result nobody actually read.

How it connects. The doctrine powers the probes of Law 7, the calibration labels of Law 10, and every external number in Part VI of this document.

Law 7: Probes, candidates, and circuit breakers

What it says. Complexity is triaged with three questions: has this exact shape succeeded here before, is it a single seam, is it cheap to undo? Two yes answers mean simple: take the direct path. Otherwise the work is complex: validate the riskiest assumption with the cheapest probe that could kill it, and when the solution space is wide, generate two or three genuinely independent candidates and judge them against explicit criteria before committing. Every step carries kill criteria written at plan time. Circuit breakers are hard: after two failed attempts on one approach, revert to the last known good state and re-diagnose; a third failure stops the work and presents attempts, hypothesis, and options. The same error twice means a different approach, never a third identical retry.

Why it exists. Sunk cost is a machine failure mode too. An agent deep in a broken approach will keep digging with perfect confidence. The breakers make stopping a rule instead of a judgment call, and pre-written kill criteria mean a disproven plan is recognized the moment it is disproven.

What it prevents. Death spirals, third identical retries, and plans that keep executing after their premise has failed.

How it connects. Triage feeds the simple-versus-complex fork of Law 0; candidate comparison uses the adversarial machinery of Law 8; clean reverts depend on the checkpointing habits of Law 5.

Law 8: Self-improvement loops

What it says. Every project runs as loops: build, gate, score, iterate. Before anything is scored, the rubric is written: the dimensions that matter, the harshest available benchmark, and what a ten means. Important findings go to independent refuters whose job is to kill them, with different lenses; majority-refuted findings die. A deterministic check is always tried before spending a model judge, because a judge costs tokens and can waffle where a command cannot. When a finding is load-bearing and a second model family is available, at least one refuter runs on it, because reviewers from one family share one family's blind spots. Checks declare severity

when written: gate or soft. The skill file itself is in scope: weaknesses are logged the moment they are seen, amendments land through a weekly consolidation under a hard size cap, every amendment names the metric it should move, and the next review reverts it if the metric did not improve. Rejected ideas are remembered with reasons and never re-proposed without new evidence.

Why it exists. A process that cannot criticize its own output plateaus at its first draft, and a process that grades itself inflates. Adversarial review with executed falsification is what makes quality claims survivable. A rulebook that grows without a validation gate becomes long, contradictory, and dead.

What it prevents. Plausible-but-wrong findings surviving review, grade inflation, and a constitution that bloats into noise.

How it connects. This is the engine room. It consumes the telemetry of Part V, drives the weekly review, applies the evidence standard of Law 15, and takes its own governance from three published sources credited in Part VII.

Law 9: Context hygiene and active forgetting

What it says. The working context is the scarcest resource and is spent like money: search before reading, read line ranges instead of whole files, never ingest raw agent transcripts, reject verbose returns at the boundary. Everything worth keeping goes to disk the moment it exists, so the conversation can be lost without losing the project. After any interruption, disk is trusted over recollection. Forgetting is active: the distilled outcome is carried forward and the journey is deliberately dropped. A short never-forget list is exempt from all forgetting: safety rules, human gates, live fences, unmerged work, open asks.

Why it exists. Long sessions degrade not because the model gets worse but because its attention fills with noise. Hygiene keeps the signal ratio high, and the disk-first habit converts session death from a catastrophe into an inconvenience.

What it prevents. The drowning coordinator, and the total loss of a crashed conversation.

How it connects. Disk-first is why the fences of Law 5 and the vault of Law 12 work; the never-forget list protects the gates of Law 11 from being optimized away.

Law 10: Honesty, calibration, and the duty to push back

What it says. Bad news travels first: a failed gate or a wrong earlier claim is reported the moment it is known, never buffered into the summary. Every claim carries its calibration where the claim is made: verified by command, verified by inspection, likely, or assumed. When the ask of the human conflicts with evidence or their own prior decisions, the session says so plainly with a recommendation, then executes their call. Anything irreversible, expensive, or strategy-changing gets a decision brief before building: two or three real options, tradeoffs, a recommendation, and the cost of a wrong call in one line.

Why it exists. A model is trained to be persuasive and agreeable, which makes uncalibrated confidence its natural register and silent compliance its natural failure. Neither improves with model quality; if anything they get more convincing.

What it prevents. The cheerful wrong answer, the buried failure, and the yes-machine that lets its principal walk into a known mistake.

How it connects. Calibration labels give the scoring of Law 15 its vocabulary; the push-back duty is measured by the challenge counter of Law 14; decision briefs route the irreversible through the gates of Law 11.

Law 11: Machine control and human gates

What it says. The session drives the machine end to end: builds, tests, browsers, native applications, screen control. Exactly one agent drives the graphical interface at any moment, with a screenshot verification after every consequential click, because the machine has one keyboard and one screen. Hard gates stay with the human, stated plainly the moment they are hit: credentials and sign-ins are never typed or automated; anything that publishes, submits, releases, or spends money requires explicit confirmation; destructive operations print exactly what they will affect and wait. When a capability is missing, the session searches for an existing tool first, proposes before installing, and builds and registers a tool only when nothing fits.

Why it exists. Autonomy without gates is how an assistant becomes a liability. The gate list is short and principled: identity, money, publication, destruction. Everything else is craft, and craft should not queue for permission.

What it prevents. Credential exposure, surprise publications, destructive commands run on pattern-match, and two agents fighting over one mouse.

How it connects. The gates implement the escalation doctrine of Law 10 physically; the single-driver rule is a concurrency cap in the spirit of Law 3.

Law 12: Structured memory: the vault

What it says. A plain folder of markdown notes is the permanent memory, with a constitution file governing how sessions read and write it. Sessions start by reading the overview of the project, its open items, and the failures index for the area at hand. They checkpoint findings at milestones, not just at the end, and close by writing a session log and updating the overview. Recall is selective: read what the task needs, never the whole vault. Hygiene mirrors active forgetting: superseded notes are archived with a pointer left behind, resolved items close the same session, and every note links to what it builds on, supersedes, or contradicts.

Why it exists. The amnesia of the model is structural, so memory must be external, durable, and legible. Plain files survive every tool change, diff cleanly in version control, and stay readable to a human at two in the morning. A memory that only grows becomes noise, so curation is a law rather than a hope.

What it prevents. The exploration tax paid every morning, and the vault that rots into a landfill nobody trusts.

How it connects. The vault stores what Law 8 learns, what Law 13 must never repeat, and what Law 14 knows about the principal.

Law 13: The known-mistakes ledger

What it says. A short list of mistakes this system has actually made, each phrased as the rule that prevents the repeat, read as law every session. Entries earn their place by having cost something real: work lost to concurrent writes, deliverables left in folders that get wiped, builds skipped after tiny edits, paths typed from memory, generated files hand-edited, headline numbers shown before their second check.

Why it exists. Experience that is not written down where the next session must trip over it is experience the system does not have. The ledger is deliberately short and blunt, because a fifty-item list is a list nobody reads.

What it prevents. Paying twice for the same lesson.

How it connects. The ledger is the fast-access tier of the failures index in Law 12, is fed by the promotion rule of Law 8, and several of its entries became full laws: the fence law and the disk-first rule both started as ledger lines.

Law 14: The principal model: think alike, think against

What it says. A living document models the human the system works for: philosophy, decision patterns, taste, a prediction ledger, and a correction log, all evidence-fed and never invented. Before presenting a significant choice, the session seals a prediction of what the principal will pick, and alignment is scored only on the briefs where prediction and recommendation diverged, because agreement cases reward telling people what they want to hear. A challenge counter runs beside the hit rate, and a period with zero challenges is flagged as a failure of the push-back duty, not as harmony. Rules distilled from corrections are born provisional and read back for confirmation. When a directive contradicts the recorded values of the principal, the session argues the case using those values as the standard, then executes the call.

Why it exists. Personalization without anti-sycophancy hardening trains a flatterer. The sealed-prediction mechanism exists because a model grading its own alignment after the fact will always find itself aligned.

What it prevents. The yes-machine, and the slow drift where the assistant optimizes for approval instead of outcomes.

How it connects. Corrections captured here become laws in the loop of Law 8; the push-back duty enforces Law 10; taste is pre-applied so work arrives already shaped, which is most of what makes the colleague posture feel like a colleague.

Law 15: Scoring every run

What it says. Every phase closes with a scorecard built from the rubric of the profile, each line naming its evidence. Self-scores cap at eight out of ten: anything above requires named external validation. A verdict from a refuter counts only when it contains an executed falsification action, never reasoning alone. Any dimension scored at plan time is re-scored on the landed thing, with the gap reported. Every close includes one check the human can run in under a minute, because trust must have a handle, not just a reporter.

Why it exists. Ungated self-assessment inflates, benchmark claims soften, and plan-time optimism silently becomes the record. The cap and the evidence rule make the scorecard a measurement instead of a mood.

What it prevents. Grade inflation and the quiet gap between what was promised and what landed.

How it connects. The scorecard consumes evidence from Laws 5 and 10, feeds the weekly review of Law 8, and its under-a-minute check is the human-facing end of the whole trust chain.

Part IV: How the laws work together

4.1 Three loops at three speeds

The fifteen laws implement three nested feedback loops. Remove any one and the system degrades in a predictable way.

```
MINUTES LOOP (inside one session)          Laws 0, 5, 10, 15
act -> verify with a command -> report with evidence
    |
    +--- fence protects the files -----+

DAYS LOOP (between sessions)                Laws 12, 13
close: write session log, open items, failures
    |
    +--- vault carries it forward -----+

WEEKS LOOP (across the record)              Law 8
telemetry accumulates -> weekly review -> ONE bounded
amendment -> next review keeps it or REVERTS it
    |
    +--- the rulebook improves or rolls back +
```

No minutes loop and you cannot trust today's work. No days loop and you pay the exploration tax forever. No weeks loop and the whole thing fossilizes at its first draft.

4.2 A worked example: one release, traced through the laws

Suppose you type: /brothermode ship version 2.1. Here is the machinery, end to end.

The session classifies the work as a product build, complex (Laws 1 and 7), and puts on the architect, safety, and project-lead hats (Law 2). It reads the vault: the overview, the open items,

and the failures index, which warns that this project once shipped a broken build from an untested one-line change (Laws 12 and 13). It maps the ground: nothing else is writing, disk is fine, the documented build and test commands are copied verbatim from the project's own files (Laws 0 and 6). The plan gets phases with kill criteria; suppose one phase needs three modules audited in parallel: three fences are registered, disjoint, each with its files, tier, and done-check, and three helpers launch as one wave (Laws 3 and 5).

One auditor claims a module is clean; the claim is re-verified against the actual files before acceptance, and one load-bearing finding goes to refuters instructed to kill it, a deterministic search first, model judges only for what a command cannot decide (Laws 5 and 8). The test suite passes, the release notes are drafted, and then the machinery stops: publishing is a hard gate, so the session states exactly what will ship and waits for your word (Law 11). After your yes and the landing, a post-release health check is scheduled (Law 8), the scorecard is written with evidence per line and a plan-versus-landed gap check (Law 15), the vault gets the session log and updated open items (Law 12), and the telemetry hook records what the whole thing actually cost without being asked (Part V). If the session had crashed anywhere in the middle, the fences, the state file, and the vault would have let the next session resume in minutes (Laws 5 and 9).

4.3 The interlock principle

Notice the pattern: no law works alone, and most failures are prevented twice. Work loss is prevented by fences and again by disk-first state. Overconfidence is prevented by done-checks, again by the self-score cap, and again by refuters. Drift is prevented by the frozen rubric and again by the validation gate on amendments. This redundancy is deliberate: single-point protections fail silently, and the cost of a second layer is small next to the cost of the failure it catches. When you extend the system, preserve the pairs.

Part V: The code, file by file

The repository is small on purpose: two Python scripts, one shell script, and a set of markdown files. No packages to install, no server, no database. Everything below is checkable by opening the file named.

5.1 The repository layout

SKILL.md	the constitution: 15 sections, ~470 lines
DIGEST.md	12-line compression, injected every session start
RUBRIC.md	template for the 9 frozen review metrics
STATE.template.md	per-project state file format (fences live here)
tools/bm_telemetry.py	the mechanical half of the learning loop
tools/bm_score.py	code-graded weekly checks
tools/bm_sessionstart.sh	session-start hook (digest + nags)
tools/WEEKLY-REVIEW.md	the weekly review procedure
docs/SETUP.md	install, hooks, vault, first week
docs/HOW-IT-WORKS.md	the mechanics in prose
vault-template/	ready-made Obsidian-compatible vault
README.md	the short version
SECURITY.md	what is stored, how to purge, how to report
LICENSE	MIT

5.2 SKILL.md and DIGEST.md: law and its shadow

SKILL.md is what a session reads when you type `/brothermode`. But sessions lose context: long conversations get compacted, new sessions start cold. So the law has a shadow: DIGEST.md, twelve lines carrying the caps, the safety floor, the never-forget list, and the wait rules. The session-start hook injects the digest into every new session automatically, which means even a session that never invoked the skill knows the non-negotiables. The law also instructs sessions to re-read its critical sections after any compaction, from disk, because laws must live in files, not in recollection. This two-tier design is how a 470-line constitution stays present without occupying 470 lines of every context window.

5.3 bm_telemetry.py: the measurement engine

660 lines of dependency-free Python, with one law enforced at its entry point: it never blocks work. Every code path, including crashes, exits cleanly; a broken hook may cost a telemetry line, never a work session. Here is the write path, verbatim, because it carries three deliberate decisions:

```
def atomic_append(path, obj):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    line = (json.dumps(obj, separators=(",", ":")) + "\n").encode()
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_APPEND, 0o644)
    try:
        os.write(fd, line)
    finally:
        os.close(fd)
```

One line, one write, append mode: two sessions ending at the same moment cannot interleave a record. The directory is created on demand, so a fresh install never fails on a missing folder. And the file descriptor closes in a finally block, so a crash mid-write cannot leak it. Small, but this function is the base of every number the learning loop later trusts.

The most important subcommand is the session-end hook target:

- **outcomes-append** parses the finished session's transcript and appends one record. Three subtleties: token counts are taken per message identity, keeping the maximum seen per message, because the harness flushes usage progressively and summing every flush would

double-count; human messages are counted only from real user turns, filtering out tool results and injected reminders; and helper-agent transcripts are parsed separately so their spend is attributed rather than lost.

- **Three guards** protect the ledger: an activity floor, so a trivial session is not recorded; a duplicate-flush guard, because the hook can fire more than once per session and an identical metric set must never be appended twice; and the atomic append above.
- **The correction scanner** runs in the same pass: short human messages matching correction patterns are captured as candidates, capped at five per session, deduplicated, for the weekly review to filter by hand. A pattern match cannot tell a correction from a quotation, so candidates are never auto-promoted to rules.
- **The rest of the toolbox:** scorecard renders the nine-metric dashboard; rate records a felt-outcome rating; startup-nags implements the overdue-review nag; stop-warn warns without ever blocking; registry-check and fence-lint audit fence hygiene; prediction-audit counts sealed predictions; migrate and dedup are maintenance, each backing up before rewriting and rechecking line counts after.

Records use OpenTelemetry attribute names for token counts, with an explicit label recording that counts are as-flushed and may lag the final turn. Absent data is absent, never estimated. Vault and registry locations are environment-configurable, and there are no writes outside the vault, which two adversarial reviews checked specifically.

5.4 `bm_score.py` and the nine metrics

The weekly review could ask a model to score everything; it deliberately does not. Ten checks are graded by code, each printing PASS, FAIL, or NO-DATA with its evidence, as shown in Part 0. The closing line states the philosophy: the model judge scores only the residue that code cannot decide. A check without data says NO-DATA, never PASS.

Those checks feed nine frozen metrics in RUBRIC.md, which the review scores against and which most teams never see spelled out, so here they are as the file names them: self-learning, token economy, speed, founder alignment, memory write-back, verification honesty, coordination, delivery, and cache efficiency. The baselines shipped are one machine's measured history. The instruction in the file is explicit: ratify your own version with whoever plays the founder role, then freeze it, because a rubric that drifts cannot measure drift.

5.5 The hooks: three wires that make it mechanical

Three entries in the settings file make the loop involuntary: session start injects the digest, session end records telemetry, and stop warns when a substantial day ends with no session log. The exact configuration is in Appendix C. The principle: anything the learning loop depends on must not depend on the model remembering to do it. The model writes the human-readable record; the hooks write the numbers; neither can fake the other.

5.6 What a vault note actually looks like

Descriptions of memory systems are abstract, so here is a real session log, lightly trimmed, of the kind written automatically at every close:

```
---
type: session-log
project: brothermode
created: 2026-07-24
verified-by: release commit 3ee9a93 published; unauthenticated ls-remote
             confirms HEAD == main; object-store sweep returns 0 forbidden hits
---

# 2026-07-24: the public release, scrubbed and triple-checked

Founder asked for a public, open-source version scrubbed of all private
data. Method: assert-exactly-once replacement script over 28 sites, so a
moved pattern aborts loudly instead of leaking silently. Then one
mechanical sweep plus two adversarial refuters briefed to REFUTE
cleanliness. Both refuted, correctly: privacy lens caught an undefined
code name and a leftover private path fingerprint; security lens caught
a /tmp symlink hazard and a migrate command whose error message promised
a backup the code never made. All fixed, re-verified by command.

Lesson: an amended commit is not a scrubbed repo. Sweep the object
store, not the working tree.
```

Note what the format enforces: a verified-by line at the top naming the evidence, the method rather than just the outcome, and one distilled lesson at the end. A session six months later reading this does not need the conversation that produced it.

5.7 bm_autosave.sh: work that survives token-death

This one is newer than the rest, and it exists because of a specific, honest failure: every rule in the constitution about not losing work (disk-first, checkpoint at green, "any kill is resumable") is prose the model must remember to run, and the moment work is most likely to be erased, running out of tokens or failing over and over, is exactly the moment the model is least able to remember anything. The rule "save before you die" was being executed by the actor that is dying.

Claude Code fires a PreCompact hook right before it compacts context, which is what happens when you run low on tokens. bm_autosave.sh hangs on that hook and snapshots your entire working tree, untracked (new) files included because those are the most-likely-lost work, into a private git ref named refs/brothermode/autosave. Three properties make it safe:

- **Non-invasive.** It stages into a throwaway index (a temporary GIT_INDEX_FILE), so your real branch, your index, and your working tree are never touched. Only a private ref moves.
- **Local only.** It runs git on your machine and never pushes. The audited zero-network property of the project still holds. It is also a separate shell script rather than part of the telemetry tool, precisely so the telemetry tool can keep its no-subprocess guarantee.
- **Recoverable.** A save with no restore is a half-feature, so bm_autosave.sh recover prints exactly how to bring the snapshot back, and after a compaction the session-start hook points

the resumed session at that command. An opt-in continuous mode also snapshots every N tool calls for a crash that is not a compaction.

Honest limit: even this does not catch a hard kill with no compaction at all, which only an external daemon would, and a daemon breaks the ten-minute, no-standing-infrastructure install. So the autosave covers the common case (token-death via compaction) mechanically and names the rare uncovered case rather than pretending to close it.

There is a deeper point the git snapshot alone misses, and it is worth stating because it took a second pass to see. Saving your files is not the whole rescue. When a session dies and resumes, the files come back but the reasoning does not: the plan, the decisions, the "where was I and why" all lived in the erased context, so a resumed model still has to reconstruct the thread. Losing the files was never the founder's real pain; losing the momentum was. Two small companions close that gap. A resume brief, written by the same PreCompact hook, distills the dying transcript's tail (the last instruction, the recent decisions, the recent commands) so the session-start hook can hand the thread back on resume. And a write-ahead intent log inverts the recording order: instead of writing a checkpoint after a milestone, where death eats the un-recorded stretch, the session logs its next intent before a risky action, so a death always leaves a forward-looking record. It is the write-ahead log pattern that has made databases crash-safe for forty years, applied to agent reasoning. Files, thread, and next intent together turn a resumed session from a restart into a continuation.

Part VI: Security, data flow, and cost

For anyone in a regulated environment, or anyone holding a budget, this part comes before the philosophy. Every external claim here carries its source, checked on July 24, 2026. Verify before you present it internally: these policies and prices change, and the research law in this very system forbids trusting a remembered price.

6.1 Where your data goes, layer by layer

The most important thing to understand is that there are two separate layers, and only one of them transmits anything.

Layer	What leaves your machine	Your control
BrotherMode itself	Nothing over any network. It writes local only: session counts, redacted correction excerpts, and (on compaction) a git snapshot of your work to a private ref	You can read the code; purge the excerpts and inspect or drop the autosave ref anytime
Claude Code (the agent)	Your prompts and the file contents it reads, over TLS 1.2 or higher, to your chosen provider	Choose the provider: Anthropic API, Bedrock, Google Cloud, or Microsoft Foundry

Model training	Nothing under commercial terms unless you opt in; consumer plans train only when you switch it on	Team, Enterprise, API, and cloud accounts are commercial by default
Retention	30 days standard for commercial accounts; Zero Data Retention available to qualified Enterprise accounts	Local transcripts also sit in plaintext on your disk for 30 days by default
Operational telemetry	Metrics, which Anthropic states never include code, prompts, or file paths. Error reports are separate and carry redacted stack traces	Separate switches per flow; one broad switch for non-essential traffic, with the web-fetch hostname check as the residual exception

6.2 What BrotherMode itself does, precisely, including the awkward part

The skill is three small scripts and a set of text files. It makes no network calls of any kind: no analytics, no phone-home, no account, no server. Nothing it touches leaves your machine. This is not a policy promise; it is a property of 834 lines of Python and shell, and you can confirm it in one command:

```
grep -rnE "urllib|requests|socket|http|curl|wget|subprocess" tools/
```

Now the part an earlier draft of this document got wrong, corrected here because a reviewer proved it wrong with a test rather than an argument. It does write two new files inside your vault, and one of them contains your words.

- **outcomes.jsonl** holds per-session counts: tokens, tool calls, helpers spawned, duration, and the folder name of the project. No file contents and no prompts.
- **corrections.jsonl** holds short excerpts of your own messages that look like corrections, capped at five per session and 400 characters each, so the weekly review can turn them into rules. This is the automatic half of the learning loop, and it is also, unavoidably, a local store of things you typed.

What protects it: secret-shaped substrings are redacted before anything is written, covering API-key and token formats, private keys, JWTs, password and credential assignments including prefixed forms such as `PROD_DB_PASSWORD`, and national-identifier and card-number shapes. The file is created owner-only. One command shows what is in it and one deletes it. Removing the session-end hook disables the capture entirely, at the cost of the automatic half of the learning loop.

```
python3 tools/bm_telemetry.py purge-corrections # show what is stored
python3 tools/bm_telemetry.py purge-corrections --yes # delete it
```

What does not protect it: redaction is best-effort pattern matching, not a guarantee, and a secret phrased in a way no pattern anticipates will be stored. Treat the file as sensitive. The vault template ships a gitignore that keeps it out of version control, which matters most for the shared-vault pattern in Part IX.

So the honest version of the claim, replacing the one an earlier draft made: BrotherMode transmits nothing, and it does add one local, redacted, owner-only, purgeable record of your corrections. Whether that is acceptable is your call, and now you have the facts to make it. There is also a second-order effect in the other direction: because the vault removes re-exploration, sessions read fewer files to reach the same understanding, so less content is transmitted to the model overall, not more.

6.3 What Claude Code does, from Anthropic's documentation

The agent underneath does transmit, by necessity. From the Claude Code data usage documentation, quoted directly:

"Commercial users (Team and Enterprise plans, API, 3rd-party platforms, and Claude Gov) maintain existing policies: Anthropic does not train generative models using code or prompts sent to Claude Code under commercial terms, unless the customer has chosen to provide their data to us for model improvement."

On retention: commercial standard is a "30-day retention period", with Zero Data Retention "available to qualified accounts for Claude Code on Claude for Enterprise", enabled per organization.

On operational metrics specifically: "Metrics never include your code, prompts, or file paths." Error reports are a separate flow: they carry error messages and stack traces, with known patterns of secrets, paths, and personal information redacted before sending, which is best-effort rather than a guarantee.

Consumer plans differ and the difference matters: on Free, Pro, and Max accounts, Anthropic will train on your data when the setting is on, with a five-year retention if you allow model improvement and thirty days if you do not. If your work is commercially sensitive, the plan you choose is the control that matters most, and it is a procurement decision, not a technical one.

Three qualifications belong next to those quotes, because a security reviewer will find them anyway and it is better they come from you.

- **The training exemption has a door in it.** The commercial policy holds unless the customer opts in, for example through the Development Partner Program, and an organization admin can opt in on behalf of the whole organization. Confirm nobody has.
- **Zero Data Retention is not the Enterprise default.** Anthropic states it is available to qualified accounts and enabled per organization by your account team after confirming eligibility. Assume you do not have it until someone confirms you do.
- **Two optional paths upload far more than metrics.** The feedback commands send a copy of conversation history including code, retained for five years. And the session-quality survey, which is on by default across providers, can be followed by a prompt asking to share the transcript; answering yes uploads the conversation, subagent transcripts, and the raw session log, with source code and file contents uploaded as-is, retained up to six months.

Nothing is uploaded unless a person clicks yes, and both paths have environment variables that disable them. Tell your developers before they click, or disable them centrally.

6.4 The switches a security team will ask about

- Choose your provider: Anthropic API, Amazon Bedrock, Google Cloud, or Microsoft Foundry. On the cloud providers, operational telemetry, error reporting, and feedback reporting default to off, though session-quality surveys stay on by default everywhere. Provider choice is also where data residency is decided, so raise it with whoever owns that policy.
- Zero Data Retention, for qualified Enterprise accounts, means no server-side persistence of prompts and outputs.
- Environment variables disable optional flows individually or all at once: telemetry, error reporting, feedback commands, and session surveys each have their own switch, plus one variable that disables non-essential traffic broadly. One residual flow survives it: before fetching a URL, the web-fetch tool sends the hostname alone, not the path or contents, to Anthropic to check a safety blocklist. It has its own separate setting. On an egress-controlled network, that is the one to know about.
- Local transcripts sit in plaintext on the developer's own disk for thirty days by default and the retention period is configurable. This is worth knowing for laptop encryption and offboarding policies, and it is true with or without this skill.
- Credentials never enter the loop: the constitution forbids typing or automating sign-ins, payments, and account logins, and requires explicit confirmation before anything publishes, releases, or spends.

6.5 What it costs

The skill is free, MIT licensed, with no paid tier and nothing reported to anyone. The model underneath is what you pay for. From the Claude Code cost documentation, quoted directly:

"Across enterprise deployments, the average cost is around \$13 per developer per active day and \$150-250 per developer per month, with costs remaining below \$30 per active day for 90% of users."

Two additions specific to this system, stated so nobody is surprised. First, a constitution costs more than a bare prompt: the ground map, the verification, and the refuters are real tokens, and a session that dispatches parallel helpers costs more again, since Anthropic's documentation notes that agent teams can use several times the tokens of a standard session. Second, the memory works in the other direction: sessions that start warm read fewer files, and prompt caching means a resumed conversation costs a fraction of a cold one. Net effect on a mature project is not something this document will pretend to have measured for your case, which is precisely why the next section hands you the arithmetic instead of a claim.

6.6 Supply chain, stated plainly

Installing this puts code in a directory that your agent executes automatically at the start of every session. That deserves the same scrutiny as any other dependency, and the honest description is: a single-maintainer, MIT-licensed repository, unsigned commits, no published releases to pin, and no third-party dependencies at all. Two of those are strengths and two are risks.

- **In its favour:** standard library only, so there is no transitive dependency tree to audit, and the entire executable surface is 834 lines that one engineer can read end to end in an afternoon. Very few dependencies can be reviewed that completely.
- **Against it:** no signed tags and no release artifacts means you cannot pin cryptographically. If your policy requires pinning, clone at a specific commit, record the hash, and review the diff before every update rather than pulling blind. A fork inside your own organization is the stronger pattern for a regulated environment, and costs you nothing but a periodic merge.

Vulnerability reports go through the repository's SECURITY.md, which also documents exactly what the tool stores and how to purge it.

6.7 What this document cannot answer for you

A security review needs more than a tool description, and pretending otherwise would waste your reviewer's time. These questions are real, this document does not answer them, and most of them are about Anthropic rather than about this skill:

- Compliance evidence: SOC 2, ISO 27001, and similar artifacts for the Anthropic dependency live in Anthropic's trust centre, not here.
- Contractual: your data processing agreement, the subprocessor list, and data residency by region are procurement questions, and the provider you choose determines the answers.
- Operational: incident response for the model provider, and vulnerability disclosure for this repository, are different processes with different owners.
- Audit: the vault is a readable record, not a tamper-evident audit log. If you need attestable evidence of what an agent did, export the harness's own telemetry into your observability stack instead of relying on notes the model wrote.
- Offboarding: deleting the skill leaves the vault behind by design, because it is your knowledge. If your policy requires destruction, the vault folder and the local session transcripts that Claude Code keeps on disk are both in scope.

The reason to list these rather than gloss them: a document that answers only the easy questions gets rejected by the reviewer who notices which ones are missing.

Part VII: The benchmark, against the base model and the alternatives

7.1 Against the base model

The disclaimer that most vendor documents omit: BrotherMode adds zero intelligence. On a single self-contained question, a bare Fable or Opus session produces the same quality of reasoning, faster and cheaper, and for that use you should not run this at all. The claim is different: on multi-session, multi-agent, high-stakes work, the dominant costs are not thinking time. They are re-exploration, rework after silent losses, wrong directions pursued past their disproof, and human time spent double-checking unverifiable claims.

Dimension	Base Fable or Opus session	With BrotherMode
Raw intelligence	Identical	Identical: the skill adds none
Memory across sessions	Starts near zero	Vault read at start, written at close
Parallel agents	Collisions possible and silent	Structurally impossible via fences
Effort sizing	Per-prompt judgment	Declared tiers, scored proportionality
Verification	Whatever the prompt demands	Done-checks and evidence blocks by law
Claim honesty	Persuasive by training	Calibration labels, capped self-scores, refuters
Crash recovery	Context lost	State on disk, resume by identity
Learning	None between sessions	Ledger, corrections capture, weekly gated amendments
Cost per task	Lowest	Higher; pays above a task-size threshold (7.3)

7.2 Against the alternative you already have

The real competitor is not another framework. It is the instructions file you have already written, or the careful prompting you already do. The honest comparison:

Approach	What it gives you	What it cannot do
Careful prompting	Good results on the task in front of you	Nothing persists; every session restarts from zero; quality depends on you writing a good prompt every time
A good CLAUDE.md	Standing project instructions, loaded every session: the right build command, house style, key invariants	It is advice with no enforcement, no memory of past runs, no measurement, and no coordination between parallel agents. Nothing checks whether it was followed

CLAUDE.md plus this skill	The instructions file keeps doing its job; the constitution adds enforcement, durable memory, fence coordination, and a measured improvement loop	It costs more tokens and needs a human to run the weekly review
---------------------------	---	---

The distinction in one line: an instructions file tells the model what you want; a constitution plus telemetry tells you whether it happened. Both are worth having, and this skill does not replace your instructions file. It reads it.

7.3 The arithmetic, with your numbers

Rather than assert a productivity multiplier, here is the model. Fill in your own figures and decide.

```

H = your loaded hourly cost
R = hours per week you currently lose to:
    re-explaining context to a fresh session
    + rework after something was silently lost or wrong
    + double-checking claims you could not verify quickly
C = monthly AI spend WITH this system, which is your current
    spend plus the overhead it adds (ground maps, verification,
    refuters, and any parallel helpers). Anthropic's published
    enterprise average is $150-250 per developer per month;
    budget the upper end, and more if you run helper fleets.

monthly value recovered = R x H x 4.3
adopt when              R x H x 4.3 > C

worked example: H = 80 -> one hour/week is worth 80 x 4.3 = $344
at R = 2 h/week: 2 x 80 x 4.3 = $688 recovered
break-even: C = 150 -> R = 150/344 = 0.44 h/week
              C = 250 -> R = 250/344 = 0.73 h/week
so at H = 80 the honest threshold is roughly half an hour
to three quarters of an hour per week, not a round number.

```

The number that decides it is R, and you are the only one who can estimate it honestly. Guessing it is where this kind of model usually goes wrong, so measure instead: for one week, keep a note every time you re-explain context to a fresh session, redo work that was silently lost, or stop to verify a claim you could not take on trust. Add the minutes. That is your R, and it takes a week and costs nothing. If it lands below roughly half an hour, the overhead will not pay at the figures above and you should not adopt this.

7.4 What the measured record actually says

Intellectual honesty requires stating what has and has not been measured. There is no controlled trial where matched teams built the same product with and without this skill; a document claiming one would be lying. What exists is: mechanisms whose failure modes are documented and reproducible, which you can verify in the code; an internal audit early in the life of this system that found coordination strong and token cost high, with the return negative at that date, which prompted the validation-gated amendment loop; and the review described in Part VIII. Two of those three live in a private working vault rather than in the public repository,

so you cannot check them yourself, and that is a limitation of this evidence rather than a detail. What you can check is every claim about the code, which is why this document points at commands rather than assertions wherever it can.

7.5 Against its peers

Three public frameworks shaped the current version, each credited in the law where its idea landed:

Public harness	What it contributed here	What was deliberately not adopted
eve (Vercel)	Gate-versus-soft check severity; deterministic checks before model judges; scored checks on every rulebook change	Its runtime and channel system: BrotherMode governs sessions, it does not host agents
gstack (Garry Tan)	Cross-model review consensus; re-scoring plans against landed work; post-release health checks	The 23-persona roster: profile-driven roles stay leaner on purpose
SkillOpt (Microsoft Research)	Validation-gated rulebook edits with revert-to-best; rejected edits kept as negative feedback	Synthetic optimizer rollouts: the training trajectories here are real work sessions, not benchmark batches

The convergence is itself evidence: three independent teams arrived at checkpointed durability, bounded edits, and adversarial verification. When independent designs agree, the shape is probably load-bearing.

Part VIII: Leverage, getting the most out of it

8.1 The objection this part respects

If you build pipelines, run a warehouse, or own an architecture, you have probably watched the agent wave from a distance with a reasonable thought: I do not have time to learn this. That instinct deserves respect, because it is usually right: most tools that demand learning before they pay are not worth the learning. So this part makes the opposite assumption. You will not learn agentic work the way you learned Spark or Snowflake. There is no syntax, no certification, no prompt-engineering course worth your weekend. You delegate in plain sentences, and the constitution in Part III is precisely what makes delegation safe without study.

The honest on-ramp is also the low-cost one: do not give it your real work first. Give it the work you dread and defer, the documentation debt, the reconciliation you re-run three times before believing, the migration comparison nobody has time to write up. Keep doing what you are good at yourself. Engineers do not adopt from enthusiasm; they adopt from receipts.

8.2 You already run this discipline

Almost every law in Part III has a name you already trust. The system did not invent its guarantees; it ported yours:

Your discipline	The same discipline, here
Transaction isolation and table locks	The single-writer fence: one writer per file set, registered before the writer starts (Law 5)
dbt tests and assertions gating a build	Done-checks: a named command whose passing output is the definition of finished (Law 15)
Query history and access audit logs	Telemetry hooks recording what every session actually did, written outside the control of the model (Part V)
The catalog and the lineage graph	The vault: decisions, failures, and session logs, linked so recall follows lineage (Law 12)
CI on every merge to main	The validation gate on the rulebook itself: a change that did not improve its named metric is reverted (Law 8)
NULL semantics done right	NO-DATA over guesses: an absent measurement is reported absent, never estimated (Law 4)
Idempotent, re-runnable jobs	Hooks and tools that fail silent, exit clean, and never corrupt a ledger on a duplicate fire (Part V)
DESCRIBE and LIMIT 5 before trusting a table	Written into the analysis profile verbatim: unfamiliar tables are inspected before they are queried, and every headline number is independently second-checked before it is shown (Law 1)

Read the last row twice if you own a warehouse. The analysis profile legislates the exact habits senior data people enforce by hand: inspect before querying, second-check before showing, label the unverified number at the number. The skill does not ask you to trust the judgment of an AI. It binds the AI to yours.

8.3 Four mornings

Illustrative scenarios, constructed to show the mechanics rather than to report specific events. They are labeled as such because a document arguing that unverified numbers must be labeled cannot present invented ones as history. Section 8.4 is the opposite: a real case, with real defects, that happened.

Morning one: the reconciliation

You type: `/brothermode prod orders and the mart disagree by about 300k rows.
Find it and tell me if finance needs to care.`

The session classifies this as analysis work, which activates the data laws automatically. It inspects both tables before querying them, structure first and sample rows second, writes the reconciliation queries, and isolates the gap. Before showing you the headline number, it

recomputes it through an independent second query, because the law forbids presenting a number that has only been computed once. Anything it could not verify in the time available is labeled unverified at the number itself.

The receipt: A short findings note with every query inline and re-runnable, the root causes with row counts attributed to each, and a one-line answer to the question you actually asked. The part you did not do: the third re-run at six in the evening to convince yourself.

Morning two: the migration that needed a decision, not a meeting

You type: `/brothermode we are moving the events pipeline off the legacy schedule. Compare streaming versus micro-batch for our shape and recommend.`

Wide solution space, so the candidate law fires: two genuinely independent designs, each strengthened before comparison, judged against explicit criteria such as cost at your volumes, failure modes, migration risk, and team skills. The output is a decision brief: both options with tradeoffs, a recommendation, the written case against its own recommendation, and the cost of a wrong call in one line. Kill criteria are attached before anyone commits.

The receipt: Wrong directions die on paper instead of in month three. And because the brief seals its prediction of your choice before you make it, the system builds an auditable track record of knowing your architecture taste.

Morning three: the documentation debt

You type: `/brothermode document the ingestion pipeline properly. The wiki page is three years old.`

The session reads the actual code and configuration, writes documentation whose every claim is checkable against a file, and flags the places where the old wiki contradicts what the code now does, with the file and line for each. It does not silently fix the contradictions it found, because deciding which side is wrong is your call. It lists them and waits.

The receipt: The task you have deferred for three years, done in an afternoon you spent elsewhere, plus a short list of drift you did not know about. Near-zero risk, pure recovered time.

Morning four: the crash that cost four minutes

You type: `(nothing: the laptop rebooted mid-refactor)`

Forty minutes into a fenced refactor, the session dies. The fences name exactly which files were claimed, the state file holds the plan and the last verified step, and the vault holds every decision made so far. The next session reads disk, not memory: it maps what changed, resumes from the last evidence block, and never re-explores what was already settled.

The receipt: Recovery measured in minutes, and the recovered session provably knows everything the dead one knew.

8.4 A real case, including the part that went wrong

On the day this repository was published, the last step before making it public was a security review of the release itself. What follows actually happened and is recorded in the project vault.

Two reviewers were dispatched in parallel, each fenced to read-only, each briefed not to check the work but to refute a specific claim: that the repository contained no private data and was safe to publish. Both came back REFUTED, which is the outcome the method is designed to produce when something is wrong.

- **The privacy reviewer** found two leaks: an undefined code name that appeared four times with dates attached, meaningless to an outside reader and traceable to private session history, and a leftover hardcoded directory path that fingerprinted the private tooling of the author.
- **The security reviewer** found two genuine defects in the code: a marker file written to a predictable location in the shared temporary directory, which on a multi-user machine is a symlink hazard, and a maintenance command whose own error message told the user to restore from a backup that the code never actually created.

All four were fixed, re-verified by command, and only then was the repository published. A fifth issue surfaced afterwards, when a further review of the live repository found that one procedure referenced a filename the template did not ship, a one-word inconsistency that would have sent a new user to a file that did not exist. It was fixed and landed on the main branch about half an hour after release. A sixth arrived later still, and is described in Part VI: a reviewer proved that captured corrections were being written in cleartext, which changed both the code and the honest wording of this document.

Two things are worth taking from this. The obvious one is that adversarial review with executed checks finds real defects that self-review misses, including in the code of the system doing the reviewing. The subtler one is what the reviewers were told: refute this claim, not review this work. Reviewers asked to confirm find reasons to confirm. This is the single cheapest technique in the entire system, and it works whether or not you ever install the skill.

8.5 The operator's playbook

- **Give outcomes, not procedures.** Say what done looks like and let the classification and tier machinery size the work. Micromanaging its steps discards its best property.
- **Name the stakes when they are high.** Saying this is a release, or be thorough, legitimately escalates tier, refuter count, and gate strictness. Saying quick check does the reverse. Proportion is a dial you hold.
- **Correct out loud.** Plain corrections are captured the same day and become standing rules with the reason recorded. Ten minutes of correction in week one shapes months of behavior.

- **Rate when asked.** The felt-outcome rating takes fifteen seconds and is the only taste signal the machinery cannot compute on its own. Skipped ratings are recorded as unrated, never guessed.
- **Use the under-a-minute check.** Every close hands you one verification you can run yourself. Run it sometimes. Trust built on spot-checks you actually perform is the durable kind.

Part IX: Operations, teams, upgrades, and troubleshooting

9.1 Vaults in a team

Two patterns work, and mixing them badly is the main way teams get this wrong.

- **Personal vaults, shared project notes.** Each engineer keeps their own vault; the project space for shared work lives in the repository itself and is committed like code. This is the recommended default: no coordination cost, no merge conflicts on telemetry, and shared knowledge travels with the code it describes.
- **One shared vault in version control.** A team vault, committed, with everyone reading and writing. This compounds knowledge fastest and costs you merge discipline: the machine-written ledgers append constantly, so exclude the telemetry folder from version control or expect conflicts on every pull.

The rule that keeps either pattern healthy is the one from Part VI on memory layers: exactly one system holds the truth. Two authoritative memories will eventually disagree, and an agent with two contradicting memories is worse than an agent with one.

9.2 Upgrading, and keeping your changes

The repository is a git clone, so updating is a pull. The friction appears when you have amended the constitution locally, which the system actively encourages. Two approaches: keep your local amendments as a documented list in your vault and re-apply them after a pull, which is simple and honest for a handful of changes; or fork the repository and merge upstream, which is correct for a team that has customized the law substantially. Either way, the amendment discipline from Law 8 applies to your changes too: log the weakness first, name the metric your change should move, and let your next weekly review keep or revert it.

9.3 Troubleshooting

Four failure modes cover nearly everything reported, and the first two are the same root cause: the hooks are the part people get wrong.

- **Telemetry never appears.** Run the session-start script by hand. If it prints the digest, your paths are right and the hook wiring is wrong. If it prints nothing, the clone path is wrong.

- **The overdue-review nag never fires, or fires forever.** The nag reads the reviews ledger, so a review that was never marked stays overdue for ever. The commands you will actually use are short and worth pinning somewhere:

```
T=~/.claude/skills/brothermode/tools/bm_telemetry.py

python3 $T review-mark "week of 24 July"      # clears the overdue nag
python3 $T rate --score 4 --task "api refactor" # your 1-5 felt outcome
python3 $T scorecard                          # the 9-metric dashboard
python3 $T purge-corrections                  # see stored excerpts
python3 tools/bm_score.py                     # the 10 coded checks
```

- **The weekly checks all say NO-DATA.** Correct on a new install. If it persists after real work, the session-end hook is not firing: see the first item.
- **Fence checks report nothing.** Fence hygiene reads the registries you declare in an environment variable. Unset, it reports NO-DATA rather than guessing at paths, which is the intended behavior, not a fault.

9.4 What this is not

- Not an autonomous agent. Nothing runs unless you start a session.
- Not a replacement for code review, testing, or your own judgment. It raises the floor on what reaches you; it does not remove you from the loop.
- Not a guarantee of correctness. It is a system for making incorrectness visible and expensive to ship.
- Not a product with a vendor behind it. It is a text file, three scripts, and an MIT license.

Conclusion

Three things, and then the imperative. First, the ceiling on AI-assisted work has moved: intelligence is the commodity, and the differentiator is the discipline around it, which is why a 470-line text file can change your output more than a model upgrade. Second, memory is the compounding asset: a team whose AI starts every session warm gets faster every week, and a team whose AI starts cold pays the same exploration tax forever. Third, trust is an engineering property: built from proofs, ledgers, capped self-scores, and measurements nobody can flatter, it survives scrutiny that vibes-based trust does not.

The model brings the intelligence. The discipline is the product.

Install it, give it the task you have been dreading, and read the receipt.

Appendix A: Glossary

Term	Plain meaning
------	---------------

LLM	Large language model: software trained on text that predicts good next words, well enough to read, reason, and code. Stateless by construction: it forgets everything between conversations.
Token	The unit LLMs read and write in, roughly three quarters of a word, and the unit you pay in.
Context window	The maximum amount of conversation and file content a model can hold at once. Exceed it and something must be dropped or summarized.
Compaction	What the harness does when a conversation approaches the context limit: it summarizes older history to free space. Detail is lost, which is why this system writes everything important to disk.
Hallucination	When a model states something false with the same confidence as something true. It is a property of prediction, not a bug to be patched, which is why this system verifies instead of trusting.
Agent	An LLM given tools: it can run commands, edit files, browse, and act, not just talk.
Subagent	A helper agent launched by a main session to work on a piece in parallel, with its own separate context.
Claude Code	The workbench from Anthropic that turns Claude into an agent on your computer.
Skill	A playbook file Claude Code reads and follows. BrotherMode is a skill.
Hook	A small script the harness runs automatically at fixed moments (session start, session end), whether or not the AI remembers to.
MCP	Model Context Protocol: a standard way to plug external tools and data sources into an agent. Relevant here only because MCP servers consume context, which the token laws account for.
Vault	The memory of BrotherMode: a plain folder of linked notes plus machine-written ledgers, on your disk, under your control.
Obsidian	A free notes application that renders the <code>[[links]]</code> of the vault as a clickable knowledge graph.
Fence	A registered claim on a set of files: while one agent holds it, no other agent may write there.
Done-check	A runnable command, named in advance, whose passing output is the definition of finished.
Telemetry	Automatic measurements of what a session actually did: tokens, tool calls, agents, duration.

JSONL	A file format: one small JSON record per line, easy to append to and easy to audit.
Rubric	A frozen list of scoring dimensions and what a ten means on each, ratified before scoring starts.
Refuter	A reviewer agent whose explicit job is to disprove a finding, not to confirm it.
Zero Data Retention	An Anthropic configuration where prompts and outputs are not persisted server-side. Available to qualified Enterprise accounts, enabled per organization.
Harness	Everything wrapped around an agent to make its work reliable: rules, memory, measurements, and gates. This skill is a harness.
Ground map	The check a session runs before touching anything: what changed on disk, what other work is in flight, what state the repository is in.
Kill criteria	The observable result, written before work starts, that would mean the plan is wrong. Written in advance so the decision to stop is not made by whoever is most invested.
Prompt caching	A provider feature that makes repeated context much cheaper to re-send. It is why a resumed conversation costs a fraction of a cold one, and why this system avoids changing model or tools mid-task.
BROTHERMODE_VAULT	The environment variable that says where your vault lives. Unset, it defaults to ~/BrotherModeVault.
BROTHERMODE_REGISTRIES	An optional environment variable listing where your project state files live, so the weekly fence checks can read them. Unset, those checks report NO-DATA rather than guessing. Note it grants read access to whatever paths you name.

Appendix B: The fence contract

The exact shape of a fence line in the state file of a project:

```
- agent: <who> (sole writer, session <id>) | tier T2 | TTL <date> |
  objective: <one line> |
  files: <exact writable set> |
  output: <what done looks like: a commit, a document, a gate> |
  boundaries: <what it must not touch> |
  termination: <observable end state> |
  check: <a runnable command that proves it worked> |
```

A fence is registered before its agent launches and closes only with an evidence block: the command run and its last lines, inline. A fence past its time-to-live is treated as released. The done-check may also assert process rather than outcome, such as maximum tool calls or required call order, so an agent that overran its declared tier is caught at the boundary.

Appendix C: Hook wiring

These three entries go in ~/.claude/settings.json. Create the file if it does not exist; if it exists and already has a hooks block, merge these into it rather than replacing the block.

```
"hooks": {
  "SessionStart": [ { "hooks": [ { "type": "command",
    "command": "sh ~/.claude/skills/brothermode/tools/bm_sessionstart.sh" } ] } ],
  "SessionEnd": [ { "hooks": [ { "type": "command",
    "command": "python3 ~/.claude/skills/brothermode/tools/bm_telemetry.py outcomes-
append" } ] } ],
  "Stop": [ { "hooks": [ { "type": "command",
    "command": "python3 ~/.claude/skills/brothermode/tools/bm_telemetry.py stop-
warn" } ] } ]
}
```

Every hook fails silent and exits cleanly: a broken hook can cost a telemetry line, never a work session.

Appendix D: Frequently asked questions

Question	Answer
Will it delete my files?	Destructive operations are gated: the law requires printing exactly what will be affected and waiting for your confirmation, every time. And the harness itself asks permission before running commands unless you turn that off.
Can it reach my database?	Only if you give it credentials or a tool that already has them. The law forbids typing credentials and forbids automating sign-ins. Operating a connection you already authenticated is allowed; opening a new one with secrets is not.
Does it run while I sleep?	No. It is a session harness, not a daemon. Nothing runs unless you start it.
Does it work offline?	The tools and the vault do. The model does not: Claude Code needs network access to reach whichever provider you configured.
What if I disagree with what it did?	Say so plainly. Corrections are captured the same day and become standing rules at the weekly review, each carrying the reason. Rules born from corrections are read back once so a one-time instruction never silently becomes law.
Do I need to know Python?	No. Python 3 must be installed because the two telemetry scripts are written in it, but you never edit or run them by hand.

Can two engineers share one vault?	Yes, and Part IX explains the two patterns and their tradeoffs. The short version: shared for a team project, personal for personal memory, and never both claiming the same authority.
What if I stop using it?	Delete the skill folder and remove three hook lines. The vault stays yours: it is plain markdown that reads fine without any of this.

Sources

External claims in this document were verified against these pages on July 24, 2026. Prices and policies change: re-check before quoting them internally.

- BrotherModeUp, the repository this paper describes: github.com/khalilmaaouni/BrotherModeUp
- Claude Code data usage, the source for every training, retention, and telemetry statement in Part VI: code.claude.com/docs/en/data-usage
- Claude Code cost management, the source for the per-developer figures in Part VI: code.claude.com/docs/en/costs
- Anthropic plan pricing: claude.com/pricing
- Claude Code, Anthropic: claude.com/claude-code
- Introducing eve, Vercel, June 2026: vercel.com/blog/introducing-eve and eve.dev/docs
- gstack, Garry Tan, 2026: github.com/garrytan/gstack
- SkillOpt, Microsoft Research, May 2026: arxiv.org/abs/2605.23904
- Obsidian: obsidian.md

Created by Khalil Maaouni.